

분류 - Part 2

✅ 부스팅 (Boosting)

1. 약한 학습기들을 순차적으로 생성하며, 이전 학습기의 오류를 점차적으로 수정
2. 이전 학습기에서 잘못 예측된 데이터 포인트에 더 많은 가중치를 부여하여, 다음 학습기가 그 부분에 더 집중하도록 함
3. 모든 학습기의 예측 결과를 합산하여 최종 예측값을 만듦

🔧 예시:

- 의료 진단 시스템
환자의 증상, 병력, 검사 결과 등을 기반으로 질병을 진단함
→ 정확도가 매우 중요하므로 오류로부터 학습하는 부스팅 방식이 효과적

🔧 구현 방식:

- AdaBoost
- Gradient Boost
- 두 알고리즘의 차이점:
Gradient Boost는 경사 하강법(Gradient Descent)을 사용함
 - 오차(Error) = 실제값 - 예측값
 - 이 오차를 줄이기 위해 경사 하강법을 사용하여 다음 학습기를 학습함

사이킷런은 GBM 기반의 분류를 위해 GradientBoostingClassifier 클래스 제공

🔧 GradientBoostingClassifier

- 장점
 - 기본 하이퍼 파라미터만으로 랜덤 포레스트보다 나은 예측 성능
- 단점
 - 수행 시간 오래 걸림
 - 하이퍼 파라미터 튜닝 노력도 필요함

🔧 GradientBoostingClassifier 주요 하이퍼파라미터

하이퍼파라미터	설명
loss	경사 하강법에 사용할 비용 함수
learning_rate	학습률 (작을수록 느리게 → 예측 성능 높아질 가능성 up, 하지만 시간 오래 걸림)
n_estimators	약한 학습기 개수
subsample	각 학습기를 학습할 때 사용할 데이터 샘플 비율 (0.5 = 50% of 학습 데이터) (과적합 방지 → 1 보다 작은 값)

✅ XGBoost(eXtra Gradient Boost)

🔧 XGBoost의 장점

- 분류 알고리즘중 뛰어난 예측 성능을 나타냄
- GBM의 단점인 느린 수행 시간 및 과적합 규제를 해결함
- 병렬 학습 가능
- 나무 가지치기 로 더 이상 이득 없는 분할을 막음
- 내부적으로 매 반복마다 교차 검증 수행하고 평가 값이 최적화 되면 중간에 멈출 수 있는 조기 중단 기능이 있음

- 결손값 자체 처리

XGBoost 핵심

- 핵심 라이브러리는 C/C++로 작성 But 파이썬 패키지 제공 "xgboost"
- XGBoost 파이썬 패키지 (xgboost)
 - 파이썬 래퍼 XGBoost module (XGBoost 전용의 파이썬 패키지)
 - 사이킷런 래퍼 XGBoost module (사이킷런과 호환되는 래퍼용 XGBoost)
 - XGBClassifier: 분류를 위한 래퍼 클래스
 - XGBRegressor: 회귀를 위한 래퍼 클래스

XGBoost 설치

```
pip install xgboost
```

파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

- 위스콘신 유방암 데이터 세트:
 - 종양의 크기, 모양 등의 속성들로 → 악성 종양(malignant)인지 양성 종양(benign)인지 분류한 데이터 세트
- Process
 - Importing libraries
 - import xgboost as xgb
 - import load_breast_cancer
 - 데이터 프레임으로 변환
 - cancer_df
 - 레이블 값의 분포 확인
 - .value_counts()
 - 1값인 양성 357개, 0 값인 악성 212개
 - 80% 학습용, 20% 테스트용으로 분할
 - X_train, X_test, y_train, y_test = train_test_split(X_features, y_label, test_size=0.2)
 - 학습용에서 90%를 최종 학습용, 10% 검증용으로 분할 (검증 성능 평가와 조기 종단을 수행 하기 위함)
 - X_tr, X_val, y_tr, y_val, = train_test_split(X_train, y_train, test_size=0.1)
 - 학습, 검증, 테스트용 DMatrix(XGBoost 만의 전용 데이터 객체) 생성
 - dtr = xgb.DMatrix(data=X_tr, label=y_tr)
 - dval = xgb.DMatrix(data=X_val, label=y_val)
 - dtest = xgb.DMatrix(data=X_test, label=y_test)
 - 하이퍼 파라미터 선정
 - params = {'max_depth':3, 'eta': 0.05, 'objective': 'binary:logistic', 'eval_metric':'logloss'}
 - num_rounds = 400
 - 모델 학습
 - 평가용 데이터 세트 설정
 - eval_list = [(dtr,'train'),(dval,'eval')] OR eval_list = [(dval,'eval')]
 - 학습

- `xgb_model = xgb.train(params=params, dtrain=dtr, num_boost_round=num_rounds, early_stopping_rounds=50, evals=eval_list)`
 - `early_stopping_rounds=50`
 - 1부터 400회까지 부스팅(`num_rounds=400`)을 반복하다가 50회를 반복하는 동안 학습 오류가 감소하지 않으면 더 이상 부스팅을 진행하지 않고 종료
 - 출력 결과 분석:
 - 126번째 반복에서 검증 데이터에 대한 `logloss` 값이 가장 낮게 나왔고, 이후 126~176번까지 `early_stopping_rounds`로 지정된 50회 동안 `logloss`값이 향상되지 않았기 때문에 멈춤
- 예측
 - `pred_probs = xgb_model.predict(dtest)` # 예측 확률 값으로 표시됨
 - `preds = [1 if x>0.5 else 0 for x in pred_probs]` # 예측 확률로 예측값 결정
- 평가
 - `get_clf_eval(y_test, preds, pred_probs)`
 - 정확도 약 0.95로 측정됨
- `plot_importance()`: xgboost 패키지에 내장된 시각화 기능 수행
 - `fig, ax = plt.subplots(figsize=(10,12))`
`plot_importance(xgb_model, ax=ax)`
- `xgboost.to_graphviz()`으로 규칙 트리 구조 시각화 가능
- `xgboost.cv()` : 교차 검증 수행후 최적 파라미터 추출 (사이킷런의 `GridSearchCV`와 유사)
 - `xgboost.cv(params, dtrain, num_boost_round=10, nfold=3, stratified=False, metrics=(), early_stopping_rounds=None)`

사이킷런 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

- Process : 조기 중단 없이 설정
 - 학습
 - `xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3, eval_metric='logloss')`
`xgb_wrapper.fit(X_train, y_train, verbose=True)` # 검증 데이터로 분할전 학습용 사용
 - 예측
 - `w_preds = xgb_wrapper.predict(X_test)`
 - `w_pred_proba = xgb_wrapper.predict_proba(X_test)[:,:1]`
 - 평가
 - `get_clf_eval(y_test, w_preds, w_pred_proba)`
 - 정확도 약 0.96로 측정됨 → 파이썬 래퍼 XGBoost보다 좋은 평가 결과
 - 이유: 건수 작은 데이터 세트에서 검증 데이터로 다시 분리하면서 더 작아지기 때문 (`X_train` → `X_tr` & `X_val`)
- Process : 조기 중단 값 50으로 설정
 - 학습
 - `xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3, eval_metric='logloss')`
`eval_list = [(X_tr, y_tr), (X_val, y_val)]` # 평가용 데이터 세트 설정
`xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_set=evals, verbose=True)`

- 예측
 - `ws50_preds = xgb_wrapper.predict(X_test)`
 - `ws50_pred_proba = xgb_wrapper.predict_proba(X_test)[:,:1]`
 - 평가
 - `get_clf_eval(y_test, ws50_preds, ws50_pred_proba)`
 - 정확도 약 0.95로 측정됨 → 파이썬 래퍼 XGBoost와 동일함
 - Process : 조기 중단 값을 10으로 설정
 - 학습
 - `xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=10, eval_set=evals, verbose=True)`
 - 예측
 - `ws10_preds = xgb_wrapper.predict(X_test)`
 - `ws10_pred_proba = xgb_wrapper.predict_proba(X_test)[:,:1]`
 - 평가
 - `get_clf_eval(y_test, ws10_preds, ws10_pred_proba)`
 - 정확도 약 0.94로 측정됨
 - 이유: 93번째 까지 수행하고 10번 반복하는 동안 성능 평가 향상되지 못해 103번째 학습 종료 → 조기 중단값을 급격하게 줄이면 예측 성능 저하될 수 있음
 - `plot_importance()` 수행 가능
-

✅ LightGBM

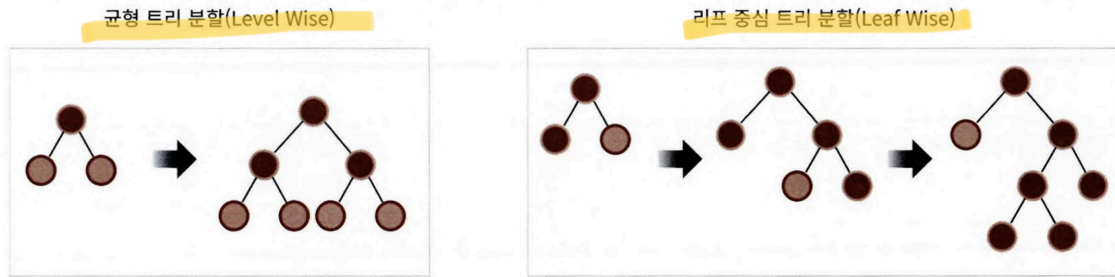
- XGBoost가 부스팅 계열 알고리즘들중 빠른 알고리즘이지만 다른 알고리즘들과 비교 할때 여전히 학습시간이 오래 걸림 → LightGBM 개발함

🔧 LightGBM의 XGBoost 대비 장점

- 학습과 예측 수행 시간 더 빠름
- 메모리 사용량 더 적음
- 카테고리형 피처의 자동 변환과 최적 분할
- 기능상의 다양성이 약간 더 많음
- 대용량 데이터에 뛰어난 성능 제공
 - 유일한 단점: 작은 데이터 세트(10,000건 이하)에 적용할 경우 과적합이 발생하기 쉬움

🔧 LightGBM의 특징

- GBM 계열의 트리 분할 방법(Level Wise): 트리의 깊이를 효과적으로 줄이기 위한 균형 트리 분할 → overfitting에 강함 but 시간 오래 걸림
- LightGBM(Leaf Wise): 리프 중심 트리 분할, 트리의 균형을 맞추지 않고, 최대 손실 값(max delta loss)을 가지는 리프 노드를 지속적으로 분할하면서 비대칭 트리가 됨, 예측 오류 손실 최소화 가능



🔧 LightGBM의 핵심

- LightGBM's 파이썬 패키지 (lightgbm)
 - 파이썬 래퍼 LightGBM module (LightGBM 전용의 파이썬 패키지)
 - 사이킷런 래퍼 LightGBM module (사이킷런과 호환되는 래퍼용 LightGBM)
 - LGBMClassifier : 분류를 위한 래퍼 클래스
 - LGBMRegressor : 회귀를 위한 래퍼 클래스

🔧 LightGBM 설치

```
pip install lightgbm
```

🔧 사이킷런 래퍼 LightGBM 적용 - 위스콘신 유방암 예측

- Process : 조기 중단 값 50으로 설정
 - 학습
 - `lgbm_wrapper = LGBMClassifier(n_estimators=400, learning_rate=0.05)`
 - `eval_list = [(X_tr, y_tr), (X_val, y_val)]` # 평가용 데이터 세트 설정
 - `lgbm_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_set=evals, verbose=True)`
 - 예측
 - `preds = lgbm_wrapper.predict(X_test)`
 - `pred_proba = lgbm_wrapper.predict_proba(X_test)[:,:1]`
 - 평가
 - `get_clf_eval(y_test, preds, pred_proba)`
 - 정확도 약 0.95로 측정됨
- `plot_importance()` 수행 가능

✅ XGBoost(파이썬 래퍼, 사이킷런 래퍼), LightGBM(파이썬 래퍼, 사이킷런 래퍼)

	파이썬 래퍼 XGBoost	사이킷런 래퍼 XGBoost	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM
	<code>xgb.train()</code>	<code>XGBClassifier</code>	<code>lgb.train()</code>	<code>LGBMClassifier</code>
Data Format	DMatrix	numpy, pandas 사용 가능	Dataset	numpy, pandas 사용 가능
파라미터 설정	params dictionary 사용	클래스 생성자에 직 접 설정	params dictionary 사용	클래스 생성자에 직 접 설정

🔧 래퍼 4종 하이퍼 파라미터 비교

1. 주요 일반 파라미터

파라미터	설명	파이썬 래퍼 XGBoost	사이킷런 래퍼 XGBoost	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM
booster	사용할 부스터 타입 지정 (gbtree, gblinear)	사용 가능	✗	✗	✗
verbosity	0: 로그 출력, 1: 로그 조용	silent/verbosity	verbosity	verbosity	verbosity
nthread	모델 학습에 사용할 병렬 처리 스레드 수 설정	nthread	n_jobs	num_threads	n_jobs

2. 부스터 파라미터

파라미터	설명	파이썬 래퍼 XGBoost	사이킷런 래퍼 XGBoost	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM
eta	학습률 (learning rate), 작을수록 느리지만 안정적 학습	params['eta']	learning_rate	params['learning_rate']	learning_rate
num_boost_rounds	트리 반복 횟수 (n_estimators 역할)	num_boost_rounds 인자로 사용	n_estimators	num_boost_rounds 인자로 사용	n_estimators
min_child_weight	노드를 분할하기 위한 최소 가중치 총합	min_child_weight	min_child_weight	min_sum_hessian_in_leaf	min_child_weight
gamma	리프 노드를 분할하기 위한 최소 손실 감소 값	gamma	gamma	✗	min_split_gain
max_depth	트리의 최대 깊이	✓	✓	✓	✓
subsample	각 트리를 학습할 때 사용할 데이터 샘플 비율	params['subsample']	subsample	params['subsample']	subsample
colsample_bytree	트리를 만들 때 사용할 피쳐 비율	colsample_bytree	colsample_bytree	feature_fraction	feature_fraction
lambda	L2 정규화 계수 (ridge)	lambda	reg_lambda	lambda_l2	reg_lambda
alpha	L1 정규화 계수 (lasso)	alpha	reg_alpha	lambda_l1	reg_alpha
scale_pos_weight	클래스 불균형 처리 시 양성 클래스 가중치 설정	✓	✓	✓	✓

3. 학습 태스크 파라미터

파라미터	설명
objective	모델이 최소화할 손실 함수 를 지정합니다.
binary:logistic	이진 분류 (0 또는 1) 문제에 사용됩니다. 출력은 확률값
multi:softmax	다중 분류 (예: 3개 이상의 클래스) 문제에서 사용. 출력은 클래스 index
multi:softprob	softmax 와 같지만 출력이 각 클래스에 대한 확률값
eval_metric	평가 지표(metric). 검증 데이터의 성능 측정에 사용됩니다.
예: rmse, mae, logloss, error, merror, mlogloss, auc 등	

래퍼 4종 코드 비교

파이썬 래퍼 XGBoost

```
import xgboost as xgb
```

```
params = {
    'eta' : 0.05,
    'eval_metric'='logloss'
}
xgb.train(params, dtrain)
```

사이킷런 래퍼 XGBoost

```
from xgboost import XGBClassifier

xgb_wrapper = XGBClassifier(learning_rate=0.05,eval_metric='logloss')
```

파이썬 래퍼 LightGBM

```
import lightgbm as lgb

params = {
    'learning_rate' : 0.05,
    'eval_metric'='logloss'
}
lgb.train(params, dtrain)
```

사이킷런 래퍼 LightGBM

```
from lightgbm import LGBMClassifier

lgbm_wrapper = LGBMClassifier(learning_rate=0.05,eval_metric='logloss')
```

✅ 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

- 지금까지 하이퍼 파라미터 튜닝에서 사용한 GridSearch 방식의 단점: 튜닝해야 할 하이퍼 파라미터 개수가 많을 때 시간 오래 걸림
 - 예:

```
params = {
    'max_depth' = [10,20,30,40,50], #5
    'num_leaves' = [35,45,55,65], #4
    'colsample_bytree' = [0.5,0.6,0.7,0.8,0.9], #5
    'subsample' = [0.5,0.6,0.7,0.8,0.9], #5
    'min_child_weight' = [10,20,30,40], #4
    'reg_alpha' = [0.01,0.05,0.1], #3
}
```

GridSearch = 5*4*5*5*4*3 = 6000회

🔧 베이지안 최적화 개요

- 목적 함수 식을 모르는 함수(Black box function)에서 최대 or 최소 반환 값을 만드는 최적 입력값(하이퍼 파라미터)을 적은 시도를 통해 찾아주는 방식

```
def black_box(hyper_parameters):
    return 최대 or 최소
```

목적 함수: 모델의 검증 정확도

입력값: learning_rate, max_depth, subsample 등
목표: 정확도를 최대화하는 최적의 하이퍼파라미터 조합을 찾기

- 모든 경우를 다 시도하는 GridSearch와 달리, 이전 실험 결과 바탕으로 확률적으로 모델을 구성하고 가장 성능 좋을 것 같은 파라미터 조합을 선택적으로 탐색
- 구성하는 두 가지 중요 요소
 - 대체 모델 (Surrogate Model) : 획득 함수로부터 전달 받은 하이퍼 파라미터 값을 기반해 최적 함수 모델을 개선
 - 최적 함수 계산 할때 다양한 알고리즘 사용 : 가우시안 프로세스(Gaussian Process), Estimator(TPE: Tree-structure Parzen Estimator)
 - 획득 함수 (Acquisition Function) : 개선된 대체 모델 기반으로 최적 하이퍼 파라미터 값을 계산

베이زي안 최적화 구성 단계

1. 초기에는 입력값(하이퍼파라미터)을 몇 개 랜덤하게 선택하여 모델을 학습하고, 해당 성능(목적 함수 값)을 수집
2. 수집된 결과를 바탕으로, 목적 함수를 근사하는 대체 모델(Surrogate Model)을 생성
3. 획득 함수(Acquisition Function)는 대체 모델이 예측한 값을 기반으로, 다음에 시도할 입력값(하이퍼파라미터)을 결정
4. 선택된 입력값으로 모델을 다시 학습하고, 얻은 성능을 대체 모델에 반영하여 업데이트함
5. 3~4 과정을 반복하면서 점점 더 좋은 하이퍼파라미터 조합을 찾아감

HyperOpt 사용하기

- 베이زي안 최적화를 ML 모델의 하이퍼 파라미터 튜닝에 적용 가능한 대표 파이썬 패키지들 : HyperOpt, Bayesian Optimization, Optuna

HyperOpt 설치

```
pip install hyperopt
```

HyperOpt 주요 로직

1. 입력 변수명과 입력값의 검색 공간(Search Space) 설정
2. 목적 함수(Objective Function) 설정
3. 목적 함수의 반환 최솟값을 가지는 최적 입력값을 추출

다른 패키지와 다르게 목적 함수 변환 값의 최댓값이 아닌 최솟값을 가지는 최적 입력값을 유추

0. Importing HyperOpt

```
from hyperopt import hp
```

1. 검색 공간(Search Space) 설정

- 파이썬 딕셔너리 형태
 - 키(입력 변수명) : value(검색 공간)
- hp 모듈이 검색 공간을 다양하게 설정 가능하도록 제공하는 대표 함수들:
 - hp.quniform(label, low, high, q)
 - hp.uniform(label, low, high)
 - hp.randint(label, upper)
 - hp.loguniform(label, low, high)

- `hp.choice(label,options)`: 검색 값에 문자열 있을 경우, `options`는 리스트나 튜플 형태
 - `hp.choice('tree_criterion',['gini','entropy'])`
- # label=입력 변수명, low=최솟값, high=최댓값, q=간격
- 예: `hp.quniform('x', -10, 10, 1)` → `x`는 -10부터 10까지 1의 간격을 가지는 값들 (순차적으로 입력되지 않음)

2. 목적 함수(Objective Function) 설정

- 검색 공간을 인자로 받고, 특정 값을 반환하는 구조(숫자형 or 딕셔너리 형태)

```
from hyperopt import STATUS_OK
def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y
    return retval / return {'loss':retval, 'status':STATUS_OK}
```

3. 목적 함수의 반환 최솟값을 가지는 최적 입력값을 추출

- HyperOpt 제공 함수 : `fmin()`
- `fmin()`의 주요 인자들:
 - `fn` : 목적 함수
 - `space`: 검색 공간 딕셔너리
 - `algo`: 베이지안 최적화 적용 알고리즘, 기본적으로 `tpe.suggest`
 - `max_evals`: 입력값 시도 횟수
 - `trials`: 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용
 - `rstate`: 동일한 결과값을 가질 수 있도록 설정하는 랜덤 시드 값
- 입력 변수 `x`의 공간 -10~10, `y`의 공간 -15~15에서 목적 함수의 반환값을 ($x^2 - 20y$)로 설정했기 때문에 → `x`는 0에 가까울 수록 `y`는 13에 가까울수록 최소 반환값
- 5번의 입력값 시도

```
from hyperopt import fmin,tpe, Trials
trial_val = Trials()
best_01 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest,
max_evals=5, trials=trial_val, rstate=np.random.default_rng(seed=0))
```

Output:

best: {'x': -4.0, 'y':12.0}

- 20번의 입력값 시도

```
best_02 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest,
max_evals=20, trials=trial_val, rstate=np.random.default_rng(seed=0))
```

Output:

best: {'x': 2.0, 'y':15.0}

- 만약 GridSearch를 사용했을 경우 $21(-10\sim10) \times 31(-15\sim15) = 651$ 회의 반복이 필요할 수 도 있는 반면, 20회의 반복만으로 일정 수준의 최적값 도출

반환값 loss 확인

- Trials 객체의 중요 속성

- results : 함수의 반복 수행 마다 반환되는 반환값
- vals: 함수의 반복 수행 마다 입력되는 입력 변수값
- results

```
print(trial_val.results)
```

Output:

- 형태 : [{'loss': 함수 반환값, 'status': 반환 상태값}, {}, ..]

```
[{'loss': -64.0, 'status': 'ok'},
 {'loss': -184.0, 'status': 'ok'},
 ...
 {'loss': 0.0, 'status': 'ok'}
]
#max_evals = 20 → dict 20개
```

- vals

```
print(trial_val.vals)
```

Output:

- 형태 : [{'loss': 함수 반환값, 'status': 반환 상태값}, {}, ..]

```
{ 'x': [-6.0,... ], 'y': [5.0,...]}
#x → 20개, y → 20개
```

- 보기 편하게 DataFrame으로 만들기

```
losses = [loss_dict['loss'] for loss_dict in trial_val.results]
result_df = pd.DataFrame({'x':trial_val.vals['x'], 'y':trial_val.vals['y'],
                          'losses':losses})
```

Output:

	x	y	losses
0	-6.0	5.0	-64.0
1	-4.0	10.0	-184.0
2	4.0	-2.0	56.0
...			
17	-4.0	-8.0	176.0
18	7.0	11.0	-171.0
19	-0.0	-0.0	0.0

HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

- 주의 할 부분
 - HyperOpt의 입력값과 반환값 모두 실수형이기 때문에 형변환 필수
 - HyperOpt의 목적 함수는 최솟값을 반환할 수 있도록 최적화하는 게 목표 → 성능 값이 클수록 좋은 성능 지표(정확도)같은 경우 -1을 곱해 줘야 함

1. 검색 공간(Search Space) 설정

```

xgb_search_space = {'max_depth': hp.quniform('max_depth',5,20,1),
                    'min_child_weight': hp.quniform('min_child_weight',1,2,1),
                    'learning_rate': hp.uniform('learning_rate',0.01,0.2),
                    'colsample_bytree': hp.uniform('colsample_bytree',0.5,1),
                    }

```

- max_depth & min_child_weight : 정수형이기때문에 hp.quniform() 사용 가능

2. 목적 함수(Objective Function) 설정

```

def objective_func(search_space):
    # 정수형 변환
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           learning_rate=search_space['learning_rate'],
                           colsample_bytree=search_space['colsample_bytree'],
                           eval_metric='logloss')
    accuracy = cross_val_score(xgb_clf, X_train, y_train, scoring='accuracy', cv=3)
    # -1으로 곱하기
    return {'loss':-1 * np.mean(accuracy), 'status': STATUS_OK}

```

3. 목적 함수의 반환 최소값을 가지는 최적 입력값을 추출

```

trial_val = Trials()
best = fmin(fn=objective_func,
           space=xgb_search_space,
           algo=tpe.suggest,
           max_evals=50,
           trials=trial_val, rstate=np.random.default_rng(seed=9))

```

Output:

colsample_bytree : 0.54241, learning_rate: 0.12601, max_depth: 17.0, min_child_weight: 2.0 → 다 실수형으로 도출

4. 도출된 하이퍼 파라미터들로 재학습과 성능 평가

```

xgb_wrapper = XGBClassifier(n_estimators=400,
                            learning_rate=round(best['learning_rate'], 5),
                            max_depth=int(best['max_depth']),
                            min_child_weight=int(best['min_child_weight']),
                            colsample_bytree=round(best['colsample_bytree'], 5)
                            )

evals = [(X_tr, y_tr), (X_val, y_val)]
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric='logloss',
               eval_set=evals, verbose=True)

preds = xgb_wrapper.predict(X_test)
pred_proba = xgb_wrapper.predict_proba(X_test)[:, 1]

get_clf_eval(y_test, preds, pred_proba)

```

Output:

- 정확도가 약 0.9649 (하이퍼 파라미터 튜닝전 보다 약간 좋은 성능)

✓ 언더 샘플링과 오버 샘플링의 이해

- 레이블이 불균형한 분포를 가진 데이터 세트를 학습시킬 때 예측 성능의 문제가 발생
 - 왜?
 - 이상 레이블 : 매우 작아서 제대로 다양한 유형을 학습 못함
 - 정상 레이블 : 매우 많아서 정상 레이블로 치우친 학습을 수행
- 불균형한 레이블 값 분포로 인한 문제 해결법
 - 언더 샘플링(Undersampling)
 - 많은 레이블을 가진 데이터 세트를 적은 레이블을 가진 데이터 세트 수준으로 감소
 - 예:
 - 정상 레이블 : 10,000건, 이상 레이블: 100건 → 정상 레이블을 100건으로
 - 오버 샘플링(Oversampling)
 - 적은 레이블을 가진 데이터 세트를 많은 레이블을 가진 데이터 세트 수준으로 증식
 - 동일한 데이터를 증식 → 과적합
 - 원본 데이터의 피쳐 값들을 약간 변경해서 증식
 - 대표적인 방법 : SMOTE
 1. 소수 클래스에서 한 데이터를 선택
 2. 그 데이터의 k-최근접 이웃(KNN)들을 구함
 3. 이웃 중 하나를 랜덤으로 선택
 4. 두 데이터 사이의 직선상 어딘가에서 새로운 데이터를 생성
- 파이썬 패키지: imbalanced-learn

✓ 스택킹 앙상블

🔧 스택킹

- 여러 개의 개별 모델의 예측 결과를 기반으로 최종 예측을 수행하는 메타 모델(Meta Model)을 학습시키는 방식
- 배깅(Bagging), 부스팅(Boosting)처럼 여러 모델을 조합하지만, 차이점은 예측값을 기반으로 다시 학습을 함

🔧 스택킹 모델 구조

- 기반 모델(Base Models): 여러 개별 모델
- 메타 모델(Meta Model): 개별 모델의 예측 결과를 입력(feature)으로 하여 최종 예측을 수행하는 모델

🔧 스택킹 Process

1. 개별 모델 학습 및 예측 수행
2. 예측 결과를 새로운 피쳐로 결합
3. 메타 모델이 이 예측 결과를 학습
4. 최종 테스트 데이터에 대해 예측 수행

🔧 스택킹 실습 요약

- 위스콘신 유방암 데이터셋 사용
- 개별 모델: KNN, 랜덤포레스트, 결정트리, 에이다부스트
- 메타 모델: 로지스틱 회귀
- 스택킹 결과: 메타 모델 정확도 97.37%, 개별 모델보다 성능 향상

CV 기반 스택킹 (Cross-Validation Stacking)

- 과적합을 줄이기 위한 방법
- 메타 모델 학습 시, 테스트 데이터의 정보 유입 방지
- KFold를 활용하여 개별 모델이 여러 번 학습과 예측을 반복하면서 메타 모델 학습/테스트 데이터를 생성

CV 기반 스택킹 구성

- 스텝 1: 각 모델별로 K-Fold를 수행하여 예측값을 수집 (train/test stacking 데이터 생성)
 - 스텝 2: 모든 모델의 stacking 데이터를 합쳐서 메타 모델 학습 및 예측 수행
 - 결과: 최종 정확도 98.25% (기본 스택킹보다 높은 성능)
-